DetectPoints 機能仕様書

1. 概要

DetectPointsは、ハイキングマップのPNG画像から円形のポイントマーカーを自動検出し、座標と名称をJSON形式で出力するコンピュータビジョンシステムです。

2. システム構成

2.1 主要コンポーネント

- **detect_points.py**: ポイント検出・抽出メインスクリプト
- **verify_points.py**: 検出結果の確認・検証スクリプト
- **requirements.txt**: 依存関係定義ファイル

2.2 依存関係

- opencv-python (>= 4.8.0): 画像処理・コンピュータビジョン
- numpy (>= 1.24.0): 数値計算
- pytesseract (>= 0.3.10): OCR（光学文字認識）
- Pillow (>= 10.0.0): 画像ファイル処理

3. detect_points.py 機能仕様

3.1 目的

ハイキングマップのPNG画像から円形ポイントマーカー（臙脂、赤、白抜き）を検出し、ポイント名を抽出してJSON出力する。

3.2 主要クラス: HikingPointDetector

3.2.1 初期化

```
HikingPointDetector(image_path: str)
```

- 画像ファイルを読み込み、基本情報を取得
- 画像が存在しない場合はValueErrorを発生

3.2.2 円形検出 - detect_circles()

- **アルゴリズム**: Hough変換による円検出
- **前処理**: ガウシアンブラー（9x9カーネル）によるノイズ除去
- **パラメータ**:
  - 最小距離: 20ピクセル
  - Cannyエッジ上閾値: 50
  - 円検出閾値: 30

    - 半径範囲: 5-30ピクセル
- **出力**: 座標(x, y)と半径rのタプルリスト

3.2.3 色フィルタリング - filter_circles_by_color()

- **対象色**:
    - 赤色系（臙脂・赤）: HSV範囲 [0-10, 170-180] H, [120-255] S, [70-255] V
    - 白色系（白抜き）: HSV範囲 [0-180] H, [0-30] S, [200-255] V
- **判定基準**:
    - 赤色ピクセル比率 > 10%
    - 白色ピクセル比率 > 30%
- **処理**: 円周辺のROI（関心領域）をHSV色空間で解析

3.2.4 テキスト抽出 - extract_text_near_point()

- **検索範囲**: 円半径の3倍の正方形領域
- **OCR設定**:
    - 言語: 日本語+英語 ('jpn+eng')
    - PSM設定: --psm 6（単一ブロックテキスト）
- **前処理**: CLAHE（制限適応ヒストグラム平坦化）によるコントラスト強化
- **フォールバック**: OCR失敗時は座標ベースの名前を生成

3.2.5 メイン処理 - detect_and_extract_points()

1. 円形ポイント検出
2. 色による絞り込み
3. 各ポイントのテキスト抽出
4. JSON構造体の生成

3.3 出力データ形式

3.3.1 JSON構造

```
{
  "image_info": {
    "filename": "画像ファイル名",
    "width": 画像幅,
    "height": 画像高さ
  },
  "points": [
    {
      "id": "ポイント名",
      "name": "ポイント名",
      "x": 絶対X座標,
      "y": 絶対Y座標,
      "relative_x": 相対X座標(0-1),
      "relative_y": 相対Y座標(0-1),
      "radius": 検出半径
    }
```

```
    ],
    "total_points": ポイント総数
  }
```

3.4 コマンドライン引数

- `input_image`: 入力PNG画像ファイル（必須）
- `-o, --output`: 出力JSONファイル名（省略時は入力名_points.json）

4. verify_points.py 機能仕様

4.1 目的

detect_points.pyの検出結果を視覚的に確認し、検証レポートを生成する。

4.2 主要クラス: PointVerifier

4.2.1 初期化

```
PointVerifier(image_path: str, json_path: str)
```

- 元画像とJSONファイルを読み込み
- データ整合性を確認

4.2.2 可視化 - draw_detected_points()

- **描画要素**:
  - 緑色の円: 検出領域の表示
  - 赤色の点: ポイント中心
  - 白色テキスト（黒背景）: ラベル・ID
- **表示オプション**:
  - show_labels: 番号ラベル表示
  - show_ids: ポイントID表示

4.2.3 レポート生成 - generate_report()

- **基本情報**: 画像ファイル名、サイズ、検出数
- **詳細情報**: 各ポイントのID、名称、座標、半径
- **統計情報**: 半径の平均・最小・最大値、座標分布範囲

4.2.4 インタラクティブモード - interactive_verification()

- **キー操作**:
  - 'l': ラベル表示切り替え
  - 'i': ID表示切り替え
  - 's': 現在表示の画像保存
  - 'r': レポート表示

    - 'q'/ESC: 終了
- **画面調整**: 最大1200x800でのリサイズ表示

4.3 コマンドライン引数

- `image`: 元PNG画像ファイル（必須）
- `json_file`: 検出結果JSONファイル（必須）
- `-o, --output`: 検証画像出力ファイル名
- `-r, --report`: レポートファイル出力先
- `--interactive`: インタラクティブモード実行
- `--show-ids`: ポイントID表示

5. 技術仕様

5.1 画像処理アルゴリズム

- **円検出**: OpenCV HoughCircles（勾配法）
- **色空間**: BGR→HSV変換による色範囲指定
- **ノイズ除去**: ガウシアンブラー
- **コントラスト強化**: CLAHE適用

5.2 OCR処理

- **エンジン**: Tesseract OCR
- **言語パック**: 日本語・英語対応
- **前処理**: グレースケール変換、コントラスト調整
- **エラーハンドリング**: 失敗時の自動フォールバック

5.3 パフォーマンス特性

- **処理対象**: 10-200ポイント/画像
- **検出精度**: 色・形状ベースのフィルタリング
- **スケーラビリティ**: 画像サイズに比例した処理時間

6. エラーハンドリング

6.1 想定エラー

- 画像ファイル読み込み失敗
- OCR処理エラー
- JSON形式不正
- メモリ不足

6.2 対応方針

- 詳細エラーメッセージの出力
- 部分的失敗時の継続処理
- フォールバック機能の提供

7. 拡張性

7.1 パラメータ調整

- Hough変換パラメータの設定可能化
- 色範囲閾値のカスタマイズ
- OCR設定の調整

7.2 機能拡張

- 他の図形（四角形、多角形）への対応
- 複数画像の一括処理
- Web APIインターフェース

8. 制限事項

8.1 技術的制限

- 円形ポイントのみ対象
- 特定色範囲への限定
- OCR精度の言語・フォント依存性

8.2 運用制限

- PNG形式画像のみ対応
- Tesseract言語パックの事前インストール必要
- 高解像度画像での処理時間増加